

✓ Named Entity Recognition on Tweets (WNUT-16)

BiLSTM + CRF (TensorFlow, word2vec embeddings) vs. fine-tuned BERT (`bert-base-uncased`)

Author: Avneesh Dubey | MS Data Science, Texas A&M University

1. Problem statement

Twitter carries about 500 million tweets a day. Twitter wants to tag and analyse tweets by **what they talk about** (people, companies, places, products, shows and so on) without depending on hashtags, which are often missing, wrong or misspelled. The underlying task is **Named Entity Recognition (NER)**: for every token in a tweet, predict whether it starts an entity (`B-`), continues one (`I-`) or is outside every entity (`O`), across 10 fine-grained types: *person, geo-loc, company, facility, product, musicartist, movie, sportsteam, tvshow, other*.

Tweets are a hard domain for NER: they are short, noisy and informally capitalised, full of @mentions, #hashtags, URLs, emoji, slang and misspellings, and they mention many new or rare entities.

Plan

1. Load the CoNLL files and explore the data (structure, label distribution, noise, out-of-vocabulary rate).
2. **Model 1: BiLSTM + CRF** in TensorFlow, with a Keras `Tokenizer` and **word2vec**-initialised embeddings, plus a set of ablations and hyperparameter runs.
3. **Model 2: fine-tune** `bert-base-uncased` with sub-word label alignment, early stopping, and several optimisers and learning rates.
4. Rejoin sub-tokens into words, evaluate with entity-level F1 (`seqeval`), save the model, and predict on new sentences.

Q1. Where can this be used, and with which modifications?

- **Trend and topic detection** on social media (the goal here): count entity mentions over time instead of relying on hashtags.
- **Brand monitoring and customer support**: find tweets that mention a company or product, and pair NER with sentiment for *aspect/entity-level sentiment*.
- **Crisis informatics**: pull out locations and facilities from disaster tweets to help direct emergency response.
- **Search, recommendation and ad targeting**: index posts by entity, and link entities to a knowledge base (*entity linking*, a natural extension).
- **Content moderation and misinformation tracking**: follow claims about specific people or organisations.
- **Modifications**: the same sequence-labelling setup, trained on other label sets, becomes biomedical NER (genes, drugs), legal and financial NER (parties, amounts, dates), PII detection and redaction, resume or invoice parsing, slot filling for chatbots, and part-of-speech tagging or chunking.

✓ 2. Setup

The brief asks for **TensorFlow 2.15**. TF 2.15 only publishes wheels for **Python ≤ 3.11**. On a Python 3.11 runtime the next cell installs it. On a newer runtime (current Colab is Python 3.12) it keeps the pre-installed TF and switches on **Keras 2 mode** (`tf_keras`, `TF_USE_LEGACY_KERAS=1`), which is the same Keras API that TF 2.15 ships. The CRF below is written in plain TensorFlow, so the code runs the same on both.

Use a GPU runtime (Runtime → Change runtime type → T4 GPU) for the BERT section.

```
import sys, subprocess, os, importlib.util, importlib.metadata as md
def pip(*pkgs):
    subprocess.run([sys.executable, "-m", "pip", "install", "-q", "--disable-pip-version-check", *pkgs], check=False)

if sys.version_info < (3, 12):
    pip("tensorflow==2.15", "tf_keras==2.15.*") # as required by the brief
else:
    tf_ver = ".".join(md.version("tensorflow").split(".")[2:])
    print(f"Python {sys.version.split()[0]}: TF 2.15 has no wheel for this version -> using pre-installed TF {tf_ver} in Keras")
    if importlib.util.find_spec("tf_keras") is None:
        pip(f"tf_keras~={tf_ver}.0") # tf_keras must match the TF minor version
    # tf.keras -> tf_keras (the Keras 2 API). transformers sets this flag itself when imported, so it is set explicitly for both
    os.environ["TF_USE_LEGACY_KERAS"] = "1"
    os.environ["TF_CPP_MIN_LOG_LEVEL"] = "3" # silence TF C++ start-up logs
    pip("gensim", "seqeval", "transformers", "accelerate")
```

```
import re, glob, random, collections, time, warnings
import numpy as np, pandas as pd
import matplotlib.pyplot as plt
```

```
# PyTorch/transformers (used for BERT) are imported BEFORE TensorFlow: TF and Triton (a PyTorch dependency) bundle
# conflicting LLVM libraries, and loading Triton after TF can crash the kernel on newer runtimes.
import torch, transformers
try:
    import triton, triton.runtime # noqa: F401
except ImportError:
    pass
from transformers import (AutoConfig, AutoTokenizer, AutoModelForTokenClassification, DataCollatorForTokenClassification,
                          TrainingArguments, Trainer, EarlyStoppingCallback, set_seed)

import tensorflow as tf
tf.get_logger().setLevel("ERROR")
for gpu in tf.config.list_physical_devices("GPU"): # let TF take GPU memory only as needed, leaving room for PyTorch/BERT
    tf.config.experimental.set_memory_growth(gpu, True)
from gensim.models import Word2Vec
from sequeval.metrics import f1_score, precision_score, recall_score, classification_report
warnings.filterwarnings("ignore")
pd.set_option("display.width", 140)

SEED = 42
random.seed(SEED); np.random.seed(SEED); tf.random.set_seed(SEED)
print("TensorFlow", tf.__version__, "| Keras", tf.keras.__version__ if hasattr(tf.keras, "__version__") else "2.x",
      "| GPU:", tf.config.list_physical_devices("GPU"))
```

3. Load the data

Each line holds `(token<TAB>label)`, and a blank line separates tweets. The files use Windows line endings (`(\r\n)`), so those are stripped. Upload the two files to the Colab working directory (or set `(DATA_DIR)`). The loader finds them by name.

```
DATA_DIR = os.environ.get("DATA_DIR", ".")
if not glob.glob(os.path.join(DATA_DIR, "*.conll")):
    try:
        from google.colab import files; files.upload() # pick both .conll files
    except ImportError:
        pass
conll = sorted(glob.glob(os.path.join(DATA_DIR, "*.conll")))
TEST_PATH = [p for p in conll if "test" in os.path.basename(p).lower()][0]
TRAIN_PATH = [p for p in conll if "test" not in os.path.basename(p).lower()][0]

def read_conll(path):
    """Return lists of token lists and tag lists; the token is the first field and the tag the last."""
    sents, tags, toks, tgs = [], [], [], []
    with open(path, encoding="utf-8") as f:
        for line in f:
            line = line.rstrip("\r\n")
            if not line.strip():
                if toks: sents.append(toks); tags.append(tgs); toks, tgs = [], []
                continue
            parts = line.split("\t") if "\t" in line else line.split()
            toks.append(parts[0]); tgs.append(parts[-1])
            if toks: sents.append(toks); tags.append(tgs)
    return sents, tags

train_sents, train_tags = read_conll(TRAIN_PATH)
test_sents, test_tags = read_conll(TEST_PATH)
print(f"train file: {os.path.basename(TRAIN_PATH)} -> {len(train_sents)} tweets, {sum(map(len, train_sents))} tokens")
print(f"test file: {os.path.basename(TEST_PATH)} -> {len(test_sents)} tweets, {sum(map(len, test_sents))} tokens")
print("\nExample tweet (token -> tag):")
print(pd.DataFrame({"token": train_sents[1], "tag": train_tags[1]}).T.to_string(header=False))
```

4. Exploratory data analysis

```
def tag_df(sents, tags, split):
    return pd.DataFrame([(i, t, g, split) for i, (s, ts) in enumerate(zip(sents, tags)) for t, g in zip(s, ts)],
                        columns=["sent_id", "token", "tag", "split"])
df = pd.concat([tag_df(train_sents, train_tags, "train"), tag_df(test_sents, test_tags, "test")], ignore_index=True)

def spans(tags):
    """BIO tags -> list of (type, start, end_exclusive). An I- without a matching B- starts a new span."""
    out, cur = [], None
    for i, t in enumerate(tags + ["0"]):
        if t == "0" or t.startswith("B-") or (t.startswith("I-") and (cur is None or cur[0] != t[2:])):
            if cur: out.append((cur[0], cur[1], i)); cur = None
            if t != "0": cur = [t[2:], i]
    return out

summary = {}
for name, S, T in [("train", train_sents, train_tags), ("test", test_sents, test_tags)]:
```

```
lens = np.array([len(s) for s in S]); ents = [e for t in T for e in spans(t)]
bad_I = sum(1 for t in T for i, g in enumerate(t) if g.startswith("I-") and (i == 0 or t[i-1][2:] != g[2:]))
summary[name] = {"tweets": len(S), "tokens": lens.sum(), "avg len": lens.mean().round(1), "max len": lens.max(),
                 "entities": len(ents), "tweets with ≥1 entity %": round(100*np.mean([len(spans(t)) > 0 for t in T]), 1),
                 "% tokens 0": round(100*np.mean([g == "O" for t in T for g in t]), 1), "invalid I- tags": bad_I}
display(pd.DataFrame(summary))
```

```
ent_counts = pd.DataFrame({n: collections.Counter(e[0] for t in T for e in spans(t))
                          for n, T in [("train", train_tags), ("test", test_tags)]}).fillna(0).astype(int)
ent_counts = ent_counts.sort_values("train", ascending=False)
ent_len = pd.Series([e[2]-e[1] for t in train_tags for e in spans(t)])

fig, ax = plt.subplots(1, 3, figsize=(17, 4))
(ent_counts / ent_counts.sum()).plot.bar(ax=ax[0], title="Entity-type share (train vs test)"); ax[0].set_ylabel("fraction of")
ax[1].hist([len(s) for s in train_sents], [len(s) for s in test_sents], bins=20, label=["train", "test"], density=True)
ax[1].set_title("Tweet length (tokens)"); ax[1].legend()
ent_len.value_counts().sort_index().plot.bar(ax=ax[2], title="Entity span length (train, tokens)")
plt.tight_layout(); plt.show()
print(ent_counts.T.to_string())
```

```
URL_RE = re.compile(r"^(https?://|www\.)\S+|^S+\.com|ly|gl|it|co|me|org|net)/\S*", re.I)
def token_kind(t):
    if URL_RE.match(t): return "url"
    if t.startswith("@") and len(t) > 1: return "@mention"
    if t.startswith("#") and len(t) > 1: return "#hashtag"
    if not any(c.isalnum() for c in t): return "punct/emoji"
    if t.isupper() and len(t) > 1: return "ALLCAPS"
    if t[0].isupper(): return "Capitalised"
    return "lowercase"

tr = df[df.split == "train"].copy(); tr["kind"] = tr.token.map(token_kind); tr["is_ent"] = tr.tag != "O"
kinds = tr.groupby("kind").agg(tokens=("token", "size"), pct_entity=("is_ent", "mean")).sort_values("tokens", ascending=False)
kinds["pct_of_tokens"] = (100 * kinds.tokens / len(tr)).round(1); kinds["pct_entity"] = (100 * kinds.pct_entity).round(1)
print("Token kinds in train (pct_entity = share of such tokens that are inside an entity):"); print(kinds.to_string())

train_vocab = set(t.lower() for s in train_sents for t in s)
test_tok = [t.lower() for s in test_sents for t in s]
test_ent_tok = [t.lower() for s, g in zip(test_sents, test_tags) for t, x in zip(s, g) if x != "O"]
print(f"\nTrain vocabulary (lower-cased): {len(train_vocab)} types")
print(f"Test-token OOV rate: {np.mean([t not in train_vocab for t in test_tok]):.1%} | "
      f"OOV rate among test ENTITY tokens: {np.mean([t not in train_vocab for t in test_ent_tok]):.1%}")
print("\nMost frequent entities per type (train):")
ent_txt = collections.defaultdict(collections.Counter)
for s, t in zip(train_sents, train_tags):
    for typ, a, b in spans(t): ent_txt[typ][" ".join(s[a:b])] += 1
for typ in ent_counts.index: print(f" {typ:12s}", ", ".join(f"{k}{{{v}}}" for k, v in ent_txt[typ].most_common(5)))
```

Findings from the EDA

- **Heavy class imbalance:** about 95% of tokens are **O**. Plain token accuracy would therefore look excellent while telling us nothing, so all evaluation uses **entity-level precision, recall and F1** (**seqeval**), where an entity counts only if both its span and its type are exactly right.
- **Small training set:** 2,394 tweets and 1,496 entities (only 38% of tweets contain any entity), spread over 10 types. Rare types (*movie*, *tvshow*, *sportsteam*) have only about 30–50 training examples each.
- **Distribution shift:** the test file is larger than the train file and has a different type mix (for example, far more *company* and *product* mentions). About half of the test entity tokens never appear in train, so models have to **generalise to unseen words**. This favours sub-word and pre-trained models such as BERT.
- **Twitter noise:** @mentions, URLs, hashtags and emoji make up a large share of tokens. URLs are never entities, and capitalisation is informative but unreliable. For the LSTM we normalise URLs and mentions and add a small *casing feature*. BERT sees the raw text, apart from light normalisation.
- Entities are mostly 1–2 tokens long, and the BIO annotation is fully consistent (O **I-** tags without a matching **B-**).

Q2. The CoNLL / BIO format. A CoNLL file puts **one token per line**. Its columns are the token, optionally other annotations (POS tag, chunk tag and so on), and **the label in the last column**. A **blank line** marks the end of a sentence (here, a tweet). Labels follow the **BIO (IOB2)** scheme: **B-TYPE** marks the first token of an entity, **I-TYPE** marks each following token of the same entity, and **O** marks tokens outside any entity. For example, **(Disney B-facility / world I-facility)**. Because every entity starts with **B-**, two adjacent entities of the same type stay separate **(B-person B-person)** is two people, while **(B-person I-person)** is one). This turns span extraction into per-token classification.

Q3. Other NER annotation formats

Format	Tags	
IO	(I-X) O	Simplest, but cannot separate two adjac
IOB1	(I-X) O , and (B-X) only when an entity directly follows another of the same type	The original CoNLL-2003 scheme; (B-
BIO / IOB2 (this dataset)	(B-X) at every entity start	The most common scheme.

Format	Tags	
BIOES / BIOLOU	adds (E-)(L-) (end) and (S-)(U-) (single-token entity)	Marks boundaries more explicitly and of
BMEWO	(B) (M) (iddle), (E) (W) (hole), (O)	Same idea as BIOES.
Stand-off / span formats	BRAT (.ann) ((T1 person 0 5 Harry)), spaCy ((start_char, end_char, label)), JSON span lists	The text and the annotations are stored
Inline markup	XML/SGML such as (<ENAMEX TYPE="PERSON">Harry</ENAMEX>) (MUC), or Markdown-style ([Harry](PER))	Entities are marked inside the running te

5. Model 1: BiLSTM + CRF (TensorFlow)

5.1 Pre-processing for the LSTM

- **Normalisation:** URLs become (<url>), @mentions become (<user>), the (#) is stripped from hashtags (#Snapchat) becomes (snapchat), which shares a vector with the plain word), text is lower-cased and digits become (0). Lower-casing shrinks the vocabulary, so a second input gives the case information back:
- **Casing feature:** one of 11 categories (lower, Title, ALLCAPS, mixed, numeric, has-digit, punctuation, mention, hashtag, url, other), with its own small embedding.
- **Train/validation split:** the provided train file is split 90/10 into *train* and *validation*. The validation set is used for early stopping and model selection. The provided **test file is never touched** until the final evaluation.

```
def norm_token(t):
    if URL_RE.match(t): return "<url>"
    if t.startswith("@") and len(t) > 1: return "<user>"
    if t.startswith("#") and len(t) > 1: t = t[1:]
    return re.sub(r"\d", "0", t.lower())

CASES = ["<pad>", "lower", "title", "allcaps", "mixed", "numeric", "hasdigit", "punct", "mention", "hashtag", "url", "other"]
def case_feat(t):
    if URL_RE.match(t): c = "url"
    elif t.startswith("@") and len(t) > 1: c = "mention"
    elif t.startswith("#") and len(t) > 1: c = "hashtag"
    elif t.isdigit(): c = "numeric"
    elif not any(ch.isalnum() for ch in t): c = "punct"
    elif any(ch.isdigit() for ch in t): c = "hasdigit"
    elif t.islower(): c = "lower"
    elif t.isupper(): c = "allcaps"
    elif t[0].isupper() and t[1:].islower(): c = "title"
    elif any(ch.isupper() for ch in t): c = "mixed"
    else: c = "other"
    return CASES.index(c)

from sklearn.model_selection import train_test_split
idx_tr, idx_va = train_test_split(np.arange(len(train_sents)), test_size=0.10, random_state=SEED)
tr_s, tr_t = [train_sents[i] for i in idx_tr], [train_tags[i] for i in idx_tr]
va_s, va_t = [train_sents[i] for i in idx_va], [train_tags[i] for i in idx_va]
print(f"train {len(tr_s)} | validation {len(va_s)} | test {len(test_sents)} tweets")

TAGS = ["0"] + sorted({g for t in train_tags for g in t} - {"0"})
tag2id = {t: i for i, t in enumerate(TAGS)}; id2tag = {i: t for t, i in tag2id.items()}
MAX_LEN = 40 # longest tweet has 39 tokens, so nothing is truncated
print(len(TAGS), "tags:", TAGS)
```

5.2 Training a TensorFlow (Keras) tokenizer

(tf.keras.preprocessing.text.Tokenizer) builds the word→index vocabulary from the **training split only**. We set (filters='') so that punctuation and emoji are kept as tokens, because tokenisation has already been done in the CoNLL file. Index 0 is kept for padding and index 1 is (<OOV>).

```
Tokenizer, pad_sequences = tf.keras.preprocessing.text.Tokenizer, tf.keras.utils.pad_sequences

tr_norm = [[norm_token(t) for t in s] for s in tr_s]
keras_tok = Tokenizer(filters="", lower=False, oov_token="<OOV>")
keras_tok.fit_on_texts(tr_norm)
word_index = keras_tok.word_index; VOCAB = len(word_index) + 1
print(f"vocabulary size (incl. <pad>=0, <OOV>={word_index['<OOV>']}): {VOCAB}")

def encode(sents, tags=None):
    X = pad_sequences(keras_tok.texts_to_sequences([[norm_token(t) for t in s] for s in sents]), MAX_LEN, padding="post", 1
    C = pad_sequences([[case_feat(t) for t in s] for s in sents], MAX_LEN, padding="post", truncating="post")
    Y = None if tags is None else pad_sequences([[tag2id[g] for g in t] for t in tags], MAX_LEN, padding="post", value=0)
    return X, C, Y

Xtr, Ctr, Ytr = encode(tr_s, tr_t); Xva, Cva, Yva = encode(va_s, va_t); Xte, Cte, Yte = encode(test_sents, test_tags)
i = 1
print("\nraw      :", tr_s[i][:12]); print("norm      :", tr_norm[i][:12])
```

```
print("word ids:", Xtr[i][:12]); print("case ids:", Ctr[i][:12]); print("tag ids :", Ytr[i][:12])
print(f"\n<OOV> share: validation {np.mean(Xva[Xva>0]==1):.1%}, test {np.mean(Xte[Xte>0]==1):.1%}")
```

5.3 word2vec embeddings

We train a **skip-gram word2vec** model (gensim) on the normalised training tweets and use it to **initialise** the embedding layer. Vectors for the **<pad>** and **<OOV>** rows are zero and random respectively. The corpus is small (about 42k tokens), so these vectors mainly capture co-occurrence within this data and their nearest neighbours are noisy. They are a starting point that is fine-tuned during training. Setting **PRETRAINED_W2V=True** loads the 300-d GoogleNews word2vec (1.6 GB download) instead. Pre-trained vectors cover more words but were trained on news text, not tweets.

```
EMB_DIM = 100
PRETRAINED_W2V = False
if PRETRAINED_W2V:
    import gensim.downloader as api
    kv = api.load("word2vec-google-news-300"); EMB_DIM = 300
else:
    w2v = Word2Vec(sentences=tr_norm, vector_size=EMB_DIM, window=5, min_count=1, sg=1, negative=10, epochs=40, seed=SEED,
    kv = w2v.wv
rng = np.random.default_rng(SEED)
emb_matrix = rng.normal(0, 0.1, (VOCAB, EMB_DIM)).astype("float32"); emb_matrix[0] = 0
hit = 0
for w, i in word_index.items():
    if w in kv: emb_matrix[i] = kv[w]; hit += 1
print(f"embedding matrix {emb_matrix.shape}; {hit}/{len(word_index)} words initialised from word2vec")
if not PRETRAINED_W2V:
    for w in ["london", "snapchat", "monday"]:
        if w in kv: print(f" most similar to '{w}':", [x for x, _ in kv.most_similar(w, topn=5)])
```

5.4 The CRF layer and the model

A softmax output layer picks each tag **independently**, so it can produce invalid sequences such as **O I-person** or **B-company I-geo-loc**. A **linear-chain CRF** adds a learned tag-to-tag **transition matrix** (plus start and end scores) and scores the **whole tag sequence** at once:

$\text{score}(x, y) = \sum_t (E_{t,y_t} + A_{y_{t-1},y_t})$, and the loss is $-\log p(y | x) = \log \sum_{y'} e^{\text{score}(x,y')} - \text{score}(x, y)$.

The normaliser is computed with the **forward algorithm** and decoding uses **Viterbi**. (**tensorflow-addons**) (which provided (**tfa.layers.CRF**)) reached end of life in 2024, so the CRF is written here in about 40 lines of plain TensorFlow. That also makes the mechanism explicit.

Architecture: **word embedding (word2vec) $\oplus$ case embedding $\rightarrow$ dropout $\rightarrow$ (Bi)LSTM $\rightarrow$ dropout $\rightarrow$ Dense(num_tags) emissions $\rightarrow$ CRF**.

```
class CRF(tf.keras.layers.Layer):
    """Linear-chain CRF: learns transition, start and end scores; computes log-likelihood and decodes with Viterbi."""
    def __init__(self, num_tags, **kw):
        super().__init__(**kw); self.num_tags = num_tags
        init = tf.keras.initializers.RandomUniform(-0.1, 0.1)
        self.trans = self.add_weight(name="transitions", shape=(self.num_tags, self.num_tags), initializer=init)
        self.start = self.add_weight(name="start", shape=(self.num_tags,), initializer=init)
        self.end = self.add_weight(name="end", shape=(self.num_tags,), initializer=init)
    def log_likelihood(self, em, tags, mask):
        # em [B,L,T], tags [B,L] int32, mask [B,L] bool
        maskf = tf.cast(mask, em.dtype); L = em.shape[1]
        # score of the gold path
        gold = tf.reduce_sum(tf.gather(em, tags[...], batch_dims=2)[..., 0] * maskf, 1)
        gold += tf.reduce_sum(tf.gather_nd(self.trans, tf.stack([tags[:, :-1], tags[:, 1:]], -1)) * maskf[:, 1:], 1)
        lengths = tf.reduce_sum(tf.cast(mask, tf.int32), 1)
        gold += tf.gather(self.start, tags[:, 0]) + tf.gather(self.end, tf.gather(tags, lengths - 1, batch_dims=1))
        # log partition function (forward algorithm)
        alpha = self.start[None] + em[:, 0]
        for t in range(1, L):
            nxt = tf.reduce_logsumexp(alpha[:, :, None] + self.trans[None] + em[:, t, None, :], axis=1)
            alpha = tf.where(mask[:, t, None], nxt, alpha)
        return gold - tf.reduce_logsumexp(alpha + self.end[None], 1)
    def viterbi(self, em, length):
        # em [L,T] numpy, one sentence
        A, S, E = self.trans.numpy(), self.start.numpy(), self.end.numpy()
        score, back = S + em[0], []
        for t in range(1, length):
            cand = score[:, None] + A + em[t][None]; back.append(cand.argmax(0)); score = cand.max(0)
        best = [int((score + E).argmax())]
        for bp in reversed(back): best.append(int(bp[best[-1]]))
        return best[::-1]

class NERTagger(tf.keras.Model):
```

```

def __init__(self, emb_matrix, num_tags, units=128, bidirectional=True, use_crf=True, use_case=True,
              trainable_emb=True, dropout=0.5, n_layers=1):
    super().__init__()
    self.use_crf, self.use_case = use_crf, use_case
    self.emb = tf.keras.layers.Embedding(emb_matrix.shape[0], emb_matrix.shape[1], trainable=trainable_emb,
                                         embeddings_initializer=tf.keras.initializers.Constant(emb_matrix))
    self.case_emb = tf.keras.layers.Embedding(len(CASES), 10)
    self.drop_in, self.drop_out = tf.keras.layers.Dropout(dropout), tf.keras.layers.Dropout(dropout)
    mk = lambda: tf.keras.layers.LSTM(units, return_sequences=True)
    self.rnn = [tf.keras.layers.Bidirectional(mk()) if bidirectional else mk() for _ in range(n_layers)]
    self.out = tf.keras.layers.Dense(num_tags)
    self.crf = CRF(num_tags) if use_crf else None
def call(self, inputs, training=False):
    words, cases = inputs; mask = tf.not_equal(words, 0)
    x = self.emb(words)
    if self.use_case: x = tf.concat([x, self.case_emb(cases)], -1)
    x = self.drop_in(x, training=training)
    for rnn in self.rnn: x = rnn(x, mask=mask, training=training)
    return self.out(self.drop_out(x, training=training)) # emissions [B,L,T]

```

5.5 Training loop, early stopping and evaluation

We use a custom `GradientTape` loop, because the CRF loss needs the mask and the gold tags together. After every epoch the model is scored on the validation set with **entity-level F1**. Training stops when validation F1 has not improved for `patience` epochs, and the **best weights are restored**. Early stopping only starts after `min_epochs=10`, because slow-starting variants (for example, frozen embeddings) sit at F1 = 0 for their first few epochs. For the no-CRF ablation the loss is masked sparse categorical cross-entropy. **Word dropout** randomly replaces training words with `<OOV>`, so the model learns what to do with unknown words. This matters here because 21% of test tokens (after normalisation) are `<OOV>`, against 11% in validation.

```

def predict_tags(model, X, C, sents, batch=256):
    preds = []
    for s in range(0, len(X), batch):
        em = model((X[s:s+batch], C[s:s+batch]), training=False).numpy()
        for j, e in enumerate(em):
            n = min(len(sents[s + j]), MAX_LEN)
            ids = model.crf.viterbi(e, n) if model.use_crf else e[:n].argmax(-1)
            preds.append([id2tag[int(k)] for k in ids] + ["0"] * (len(sents[s + j]) - n))
    return preds

def train_lstm(cfg, epochs=40, patience=5, min_epochs=10, batch=32, verbose=False):
    tf.random.set_seed(SEED); np.random.seed(SEED)
    model = NERTagger(emb_matrix, len(TAGS), **{k: cfg[k] for k in ["units", "bidirectional", "use_crf", "use_case", "trainable_emb", "dropout", "n_layers"]})
    opt = {"adam": tf.keras.optimizers.Adam, "rmsprop": tf.keras.optimizers.RMSprop}[cfg["optimizer"]](cfg["lr"], clipnorm=1)
    model.compile(optimizer=opt, loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True, reduction="none"))

    @tf.function
    def step(x, c, y):
        mask = tf.not_equal(x, 0)
        with tf.GradientTape() as tape:
            em = model((x, c), training=True)
            if model.use_crf: loss = -tf.reduce_mean(model.crf.log_likelihood(em, y, mask))
            else:
                m = tf.cast(mask, tf.float32); loss = tf.reduce_sum(ce(y, em) * m) / tf.reduce_sum(m)
        opt.apply_gradients(zip(tape.gradient(loss, model.trainable_variables), model.trainable_variables))
        return loss

    best_f1, best_w, wait, hist = -1, None, 0, []
    for ep in range(1, epochs + 1):
        order = np.random.permutation(len(Xtr)); losses = []
        for s in range(0, len(order), batch):
            b = order[s:s+batch]; x = Xtr[b].copy()
            if cfg["word_dropout"] > 0:
                drop = (np.random.rand(*x.shape) < cfg["word_dropout"]) & (x > 1); x[drop] = 1
            losses.append(float(step(tf.constant(x), tf.constant(Ctr[b]), tf.constant(Ytr[b].astype("int32")))))
        f1 = f1_score(va_t, predict_tags(model, Xva, Cva, va_s))
        hist.append({"epoch": ep, "loss": np.mean(losses), "val_f1": f1})
        if verbose: print(f" epoch {ep:2d} loss {np.mean(losses):.3f} val_f1 {f1:.3f}")
        if f1 > best_f1: best_f1, best_w, wait = f1, model.get_weights(), 0
        else:
            wait += 1
            if wait >= patience and ep >= min_epochs: break
    model.set_weights(best_w)
    return model, pd.DataFrame(hist), best_f1

```

5.6 Experiments: bidirectionality, CRF vs softmax, features and hyperparameters

Each configuration is trained with early stopping (patience 5, at most 40 epochs). Model selection uses **validation** entity F1 only. Test F1 is also listed for every configuration, because the 240-tweet validation set is small and noisy while the 3,850-tweet test set gives a steadier comparison.

```
BASE = dict(units=128, bidirectional=True, use_crf=True, use_case=False, trainable_emb=True, dropout=0.5, n_layers=1,
            optimizer="adam", lr=1e-3, word_dropout=0.0)
EXPERIMENTS = {
    "1 Uni-LSTM + CRF": dict(BASE, bidirectional=False),
    "2 BiLSTM + softmax (no CRF)": dict(BASE, use_crf=False),
    "3 BiLSTM + CRF": dict(BASE,
    "4 BiLSTM + CRF, frozen w2v": dict(BASE, trainable_emb=False),
    "5 BiLSTM + CRF + case feats": dict(BASE, use_case=True),
    "6 #5 + word-dropout 0.1": dict(BASE, use_case=True, word_dropout=0.1),
    "7 #6, 256 units, 2 layers": dict(BASE, use_case=True, word_dropout=0.1, units=256, n_layers=2),
    "8 #6, RMSprop lr 3e-3, dropout .3": dict(BASE, use_case=True, word_dropout=0.1, optimizer="rmsprop", lr=3e-3, dropout=
}
lstm_results, lstm_hist, lstm_models = [], {}, {}
for name, cfg in EXPERIMENTS.items():
    t0 = time.time(); m, h, f1 = train_lstm(cfg)
    lstm_models[name], lstm_hist[name] = m, h
    te_f1 = f1_score(test_tags, predict_tags(m, Xte, Cte, test_sents))
    lstm_results.append({"experiment": name, "val_F1": round(f1, 4), "test_F1": round(te_f1, 4), "best_epoch": int(h.val_f1_
                        "epochs_run": len(h), "time_s": round(time.time() - t0)})
lstm_results = pd.DataFrame(lstm_results).set_index("experiment"); display(lstm_results)
```

```
fig, ax = plt.subplots(1, 2, figsize=(15, 4))
for n, h in lstm_hist.items(): ax[0].plot(h.epoch, h.val_f1, label=n.split(" ", 1)[1][:30])
ax[0].set(title="LSTM variants: validation entity F1 per epoch", xlabel="epoch", ylabel="F1"); ax[0].legend(fontsize=7)
lstm_results[["val_F1", "test_F1"]].plot.barh(ax=ax[1], title="Entity F1 per configuration"); ax[1].invert_yaxis()
plt.tight_layout(); plt.show()
```

Implications of bidirectional models. A forward-only LSTM tags token t using only tokens $1..t$. A **BiLSTM** runs a second LSTM from right to left and joins both states, so every tag is conditioned on the **whole tweet**. That matters for NER: in "*Bruins* beat the Leafs" versus "*Paris* Hilton arrived", the words that follow decide the entity type and its boundary. The costs are twice the parameters and compute, and the **whole sequence must be available before any tag is output**, so a BiLSTM cannot tag a stream token by token as it arrives. For batch tagging of complete tweets this is not a limitation.

What the experiments show. Compare 1 vs 3 (backward direction) and 2 vs 3 (CRF vs softmax) in the table. With only 240 validation tweets these architectural differences are small and partly within noise. The model is limited mainly by **unknown words**, not by context or tag structure. The large, consistent gains come from the two changes aimed at OOV words: **caseing features** (#5) let the model spot a capitalised or hashtagged word it has never seen, and **word dropout** (#6) trains the $\langle \text{OOV} \rangle$ embedding so that unseen words are handled sensibly. Frozen embeddings (#4) do worse than fine-tuned ones, because word2vec vectors trained on 2k tweets are weak (see the nearest neighbours above). Validation F1 is clearly higher than test F1 for every configuration. This reflects the train→test shift found in the EDA: the test set has twice the OOV rate and a different mix of entity types.

```
best_lstm_name = lstm_results.val_F1.idxmax(); best_lstm = lstm_models[best_lstm_name]
lstm_test_pred = predict_tags(best_lstm, Xte, Cte, test_sents)
lstm_test = {"P": precision_score(test_tags, lstm_test_pred), "R": recall_score(test_tags, lstm_test_pred), "F1": f1_score(
print(f"Best LSTM config: {best_lstm_name}\nTEST precision {lstm_test['P']:.4f} recall {lstm_test['R']:.4f} entity-F1 {
print(classification_report(test_tags, lstm_test_pred, digits=3))
```

Learned CRF transition scores for a few tag pairs confirm that the CRF has picked up the BIO grammar. Transitions such as $O \rightarrow I-x$ and $B\text{-}person \rightarrow I\text{-}company$ get low scores, while $B-x \rightarrow I-x$ gets high ones.

```
if best_lstm.use_crf:
    A = pd.DataFrame(best_lstm.crf.trans.numpy(), index=TAGS, columns=TAGS)
    show = ["O", "B-person", "I-person", "B-geo-loc", "I-geo-loc", "B-company", "I-company"]
    print("transition score row = previous tag, column = next tag\n", A.loc[show, show].round(2).to_string())
```

6. Model 2: fine-tuning BERT (bert-base-uncased)

6.1 Loading the model, config and tokenizer

The model is trained with the Hugging Face (`transformers`) library in its standard PyTorch backend (`Trainer`). The model, config and tokenizer are the same objects under TensorFlow, and `save_pretrained` writes a checkpoint that both frameworks can load.

```
from transformers.trainer_callback import PrinterCallback
from transformers.utils import logging as hf_logging
hf_logging.set_verbosity_error() # hide the expected "classifier weights newly initialised" warning
hf_logging.disable_progress_bar() # no weight-loading / saving progress bars in the printed PDF
```

```
MODEL_NAME = "bert-base-uncased"
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"

config = AutoConfig.from_pretrained(MODEL_NAME, num_labels=len(TAGS), id2label=id2tag, label2id=tag2id)
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME) # fast (Rust) WordPiece tokenizer, lower-cases the text
print(f"device={DEVICE} | layers={config.num_hidden_layers} hidden={config.hidden_size} heads={config.num_attention_heads}
      f"vocab={config.vocab_size} | num_labels={config.num_labels} | fast tokenizer: {tokenizer.is_fast}")
```

6.2 What does WordPiece tokenisation do to our data?

BERT has a fixed vocabulary of about 30k **sub-word** pieces. Any word not in that vocabulary is split into pieces (continuations are marked `##`). The tokenizer also **adds** the special tokens `[CLS]` at the start and `[SEP]` at the end, lower-cases the text and strips accents. Our labels, however, are **per word**. So after tokenising we need to check (a) how much the sequences grow, (b) which words become `[UNK]` and (c) whether any word yields **zero** sub-tokens. A word with zero sub-tokens would silently disappear and receive no prediction.

```
ex = train_sents[1]
enc = tokenizer(ex, is_split_into_words=True)
toks = tokenizer.convert_ids_to_tokens(enc["input_ids"])
print("words      :", ex[:14]); print("sub-tokens:", toks[:24]); print("word_ids  :", enc.word_ids()[:24])

def tok_stats(sents):
    n_sub, unk, empty = [], collections.Counter(), collections.Counter()
    for s in sents:
        for w in s:
            p = tokenizer.tokenize(w)
            if not p: empty[w] += 1
            elif p == [tokenizer.unk_token]: unk[w] += 1
            n_sub.append(len(p))
    lens = [len(tokenizer(s, is_split_into_words=True)["input_ids"]) for s in sents]
    return np.array(n_sub), unk, empty, np.array(lens)
n_sub, unk, empty, lens = tok_stats(train_sents)
print(f"\nsub-tokens per word: mean {n_sub.mean():.2f}; words split into ≥2 pieces: {np.mean(n_sub>1):.1%}")
print(f"sequence length with specials: mean {lens.mean():.1f}, max {lens.max()} (vs max {MAX_LEN-1} words)")
print(f"words -> [UNK]: {sum(unk.values())} e.g. {list(unk)[:8]}")
print(f"words -> ZERO sub-tokens: {sum(empty.values())} e.g. {[repr(w) for w in list(empty)[:6]]}")
print("longest splits:", sorted({w for s in train_sents for w in s}, key=lambda w: -len(tokenizer.tokenize(w)))[3:])
```

Do we need to remove or add tokens?

- `[CLS]` `[SEP]` are **added automatically**. They carry no word label, so they get the label `-100`, which the loss ignores.
- URLs** break into 10–25 meaningless pieces, inflate the sequence length and are never entities. **@mentions** are sometimes entities (such as a `@Snapchat` handle), but most are random user names. A light *BERTweet-style* normalisation is applied: every URL becomes the single word `http`, and the user-name text is kept. This shortens sequences without losing label alignment, because it is still one word in, one word out.
- Words that produce **zero sub-tokens** (invisible characters such as the variation selector `U+200D` or zero-width joiners) and emoji that the uncased vocabulary maps to `[UNK]` are replaced by `[UNK]` explicitly. This way **every word keeps at least one sub-token and therefore a prediction**, and word counts stay identical to the gold labels.
- No new vocabulary tokens are needed. Adding tokens such as `<url>` would require `resize_token_embeddings` and would start them from random vectors, which a dataset this small cannot train well.

```
def bert_norm_word(w):
    if URL_RE.match(w): return "http"
    return w if tokenizer.tokenize(w) else tokenizer.unk_token
def bert_norm(sents): return [[bert_norm_word(w) for w in s] for s in sents]

LABEL_ALL_SUBTOKENS = False # False: only the first sub-token of a word is trained and scored; the rest get -100
MAX_BERT_LEN = 128
def tokenize_and_align(sents, tags):
    enc = tokenizer(bert_norm(sents), is_split_into_words=True, truncation=True, max_length=MAX_BERT_LEN)
    all_labels = []
    for i, t in enumerate(tags):
        prev, labs = None, []
        for wid in enc.word_ids(batch_index=i):
            if wid is None: labs.append(-100) # [CLS] / [SEP] / padding
            elif wid != prev: labs.append(tag2id[t[wid]]) # first sub-token of a word
            else: # continuation sub-token
                labs.append(tag2id[t[wid].replace("B-", "I-")] if LABEL_ALL_SUBTOKENS else -100)
            prev = wid
        all_labels.append(labs)
    return [{"input_ids": enc["input_ids"][i], "attention_mask": enc["attention_mask"][i], "labels": all_labels[i]} for i in range(len(sents))]

bert_tr, bert_va, bert_te = tokenize_and_align(tr_s, tr_t), tokenize_and_align(va_s, va_t), tokenize_and_align(test_sents,
```

```
# sanity checks: the labelled positions correspond exactly to the words, in order
for ds, S, T in [(bert_tr, tr_s, tr_t), (bert_va, va_s, va_t), (bert_te, test_sents, test_tags)]:
    assert all(sum(l != -100 for l in d["labels"]) == len(s) for d, s in zip(ds, S)), "word lost in tokenisation!"
    assert all([id2tag[l] for l in d["labels"] if l != -100] == t for d, t in zip(ds, T))
d = bert_tr[1]
print(pd.DataFrame({"sub-token": tokenizer.convert_ids_to_tokens(d["input_ids"]),
                    "label": [id2tag.get(l, "-100") for l in d["labels"]]}).head(18).T.to_string(header=False))
print(f"\nAlignment checks passed: every word has exactly one labelled sub-token (train {len(bert_tr)}, val {len(bert_va)}).")
print("Max sub-token length after URL normalisation:", max(len(d["input_ids"]) for d in bert_tr + bert_va + bert_te))
```

6.3 Training: metric, experiments with optimisers, learning rates, epochs and early stopping

The train/validation/test split is the same as for the LSTM. `compute_metrics` turns logits back into **word-level** tag sequences, using only the positions whose label is not -100, which are the first sub-tokens. It then reports `seqeval` entity-level P/R/F1. Four runs are compared:

```
collator = DataCollatorForTokenClassification(tokenizer)
def logits_to_word_tags(logits, labels):
    pred = logits.argmax(-1); out_p, out_t = [], []
    for p, l in zip(pred, labels):
        keep = l != -100
        out_p.append([id2tag[int(x)] for x in p[keep]]); out_t.append([id2tag[int(x)] for x in l[keep]])
    return out_p, out_t
def compute_metrics(ev):
    p, t = logits_to_word_tags(*ev)
    return {"precision": precision_score(t, p), "recall": recall_score(t, p), "f1": f1_score(t, p)}

BERT_RUNS = {
    "A AdamW lr5e-5, 4 ep": dict(optim="adamw_torch", lr=5e-5, epochs=4, early_stop=False),
    "B AdamW lr5e-5, 20 ep, no early stop": dict(optim="adamw_torch", lr=5e-5, epochs=20, early_stop=False),
    "C AdamW lr3e-5, ≤20 ep, early stop(3)": dict(optim="adamw_torch", lr=3e-5, epochs=20, early_stop=True),
    "D Adafactor lr1e-4, ≤20 ep, early stop": dict(optim="adafactor", lr=1e-4, epochs=20, early_stop=True),
}
def run_bert(name, r, out_dir):
    set_seed(SEED)
    model = AutoModelForTokenClassification.from_pretrained(MODEL_NAME, config=config)
    args = TrainingArguments(
        output_dir=out_dir, learning_rate=r["lr"], optim=r["optim"], num_train_epochs=r["epochs"],
        per_device_train_batch_size=16, per_device_eval_batch_size=64, weight_decay=0.01, warmup_steps=int(0.1 * r["epochs"]),
        lr_scheduler_type="linear", eval_strategy="epoch", save_strategy="epoch", save_total_limit=1,
        load_best_model_at_end=True, metric_for_best_model="f1", greater_is_better=True,
        logging_strategy="epoch", report_to="none", disable_tqdm=True, seed=SEED, fp16=torch.cuda.is_available())
    trainer = Trainer(model=model, args=args, train_dataset=bert_tr, eval_dataset=bert_va, data_collator=collator,
                     compute_metrics=compute_metrics,
                     callbacks=[EarlyStoppingCallback(early_stopping_patience=3)] if r["early_stop"] else [])
    trainer.remove_callback(PrinterCallback) # keep the printed output short
    t0 = time.time(); trainer.train()
    logs = pd.DataFrame(trainer.state.log_history)
    ev = logs.dropna(subset=["eval_f1"])[["epoch", "eval_loss", "eval_f1"]].reset_index(drop=True)
    tl = logs.dropna(subset=["loss"])[["epoch", "loss"]]
    ev = ev.merge(tl, on="epoch", how="left")
    final_f1 = ev.eval_f1.iloc[-1]; best = trainer.evaluate()["eval_f1"]
    res = {"run": name, "best_val_F1": round(best, 4), "last_epoch_val_F1": round(final_f1, 4),
          "best_epoch": int(ev.epoch[ev.eval_f1.idxmax()]), "epochs_run": int(ev.epoch.max()), "time_s": round(time.time() - t0, 4)}
    print(f"{name:40s} best val F1 {best:.4f} @ epoch {res['best_epoch']} | ran {res['epochs_run']} epochs | last-epoch F1 {final_f1:.4f}")
    return trainer, ev, res

bert_results, bert_hist, best_trainer, best_f1 = [], {}, None, -1
for i, (name, r) in enumerate(BERT_RUNS.items()):
    trainer, ev, res = run_bert(name, r, f"bert_run_{i}")
    bert_results.append(res); bert_hist[name] = ev
    if res["best_val_F1"] > best_f1:
        best_f1, best_name = res["best_val_F1"], name
        if best_trainer is not None: del best_trainer
        best_trainer = trainer
    else:
        del trainer
    if torch.cuda.is_available(): torch.cuda.empty_cache()
bert_results = pd.DataFrame(bert_results).set_index("run"); display(bert_results)
```

```
fig, ax = plt.subplots(1, 2, figsize=(15, 4))
for n, h in bert_hist.items():
    ax[0].plot(h.epoch, h.eval_f1, marker=".", label=n); ax[1].plot(h.epoch, h.eval_loss, marker=".", label=n)
ax[0].set(title="BERT: validation entity F1", xlabel="epoch"); ax[1].set(title="BERT: validation loss", xlabel="epoch")
ax[0].legend(fontsize=7); plt.tight_layout(); plt.show()

hB = bert_hist[[k for k in bert_hist if k.startswith("B")]][0]
print(f"Run B (no early stopping): val loss is lowest at epoch {int(hB.epoch[hB.eval_loss.idxmin()])} and ends {hB.eval_loss.iloc[-1]:.4f}")
```

```
f"val F1 peaks at epoch {int(hB.epoch[hB.eval_f1.idxmax()])} ({hB.eval_f1.max():.3f}) and the final epoch scores {hB.
for k, r in bert_results.iterrows():
    if BERT_RUNS[k]["early_stop"]:
        print(f"Run {k[0]} (early stopping): stopped after {int(r.epochs_run)} of {BERT_RUNS[k]['epochs']} epochs "
              f"({1 - r.epochs_run / BERT_RUNS[k]['epochs']:.0%} of the budget saved), best val F1 {r.best_val_F1:.3f}.")
```

Q6. Did early stopping have an effect? (See the printout, table and curves above.) Yes, mainly on **training time and on which checkpoint we end up with**, rather than on the peak score. With only about 2k training tweets, BERT fits the training set within a few epochs. After that, validation **loss rises steadily** (over-confidence and memorisation) while validation **F1 flattens or drifts**. Without early stopping (run **B**) the extra epochs are wasted compute, and the *final* model is worse than or no better than the best epoch. It only matches the early-stopped runs because `load_best_model_at_end` goes back to the best checkpoint. Early stopping (runs **C** and **D**) ends training a few epochs after the F1 peak, skips most of the 20-epoch budget and keeps the best checkpoint, so it acts as regularisation and makes a hand-picked epoch count (run **A**) unnecessary. F1 on a 240-tweet validation set is noisy, so a patience of 3 epochs keeps one noisy epoch from stopping training too early.

6.4 Test-set evaluation and joining sub-tokens back into words

The model predicts a tag for **every sub-token**. To return to the original words we use `word_ids()`: each word takes the prediction of its **first sub-token**, and continuation (`##`) pieces, `[CLS]` and `[SEP]` are dropped. Because we ensured every word has at least one sub-token, the output has exactly one tag per input word.

```
model_bert = best_trainer.model.eval().to(DEVICE)
def bert_predict_words(sents, batch=64, show_subtokens=False):
    """sentences as lists of words -> list of word-level tag lists (first-sub-token strategy)"""
    out = []
    for s in range(0, len(sents), batch):
        chunk = sents[s:s+batch]
        enc = tokenizer(bert_norm(chunk), is_split_into_words=True, truncation=True, max_length=MAX_BERT_LEN,
                        padding=True, return_tensors="pt").to(DEVICE)
        with torch.no_grad(): pred = model_bert(**enc).logits.argmax(-1).cpu().numpy()
        for i, words in enumerate(chunk):
            wids, tags, prev = enc.word_ids(batch_index=i), [], None
            for pos, wid in enumerate(wids):
                if wid is not None and wid != prev: tags.append(id2tag[int(pred[i, pos])])
                prev = wid
            tags += ["O"] * (len(words) - len(tags)) # only if truncated (never happens here)
            out.append(tags)
        if show_subtokens and s + i == 0:
            sub = tokenizer.convert_ids_to_tokens(enc["input_ids"][i])
            n = int(enc["attention_mask"][i].sum())
            print("sub-token level:", " ".join(f"{t}/{id2tag[int(p)]}" for t, p in zip(sub[:n], pred[i][:n])))
            print("word level      :", " ".join(f"{w}/{g}" for w, g in zip(words, tags)))
    return out

bert_test_pred = bert_predict_words(test_sents, show_subtokens=True)
bert_test = {"P": precision_score(test_tags, bert_test_pred), "R": recall_score(test_tags, bert_test_pred), "F1": f1_score(
print(f"\nBest BERT run: {best_name}\nTEST precision {bert_test['P']:.4f} recall {bert_test['R']:.4f} entity-F1 {bert_t
print(classification_report(test_tags, bert_test_pred, digits=3))
```

```
comp = pd.DataFrame({"BiLSTM-CRF (" + best_lstm_name.split(" ", 1)[1] + ")": lstm_test, "BERT (" + best_name.split(" ", 1)
print("Held-out TEST set, entity-level (sequeval, exact span + type):"); display(comp)
per_type = pd.DataFrame({
    "LSTM F1": pd.Series({k: v["f1-score"] for k, v in classification_report(test_tags, lstm_test_pred, output_dict=True).
    "BERT F1": pd.Series({k: v["f1-score"] for k, v in classification_report(test_tags, bert_test_pred, output_dict=True).
per_type.loc[[k for k in per_type.index if "avg" not in k]].plot.bar(figsize=(12, 3.5), title="Per-type test F1"); plt.tigh
```

6.5 Error analysis: a few test tweets where BERT is wrong

```
shown = 0
for s, g, p in zip(test_sents, test_tags, bert_test_pred):
    if spans(g) != spans(p) and 6 <= len(s) <= 20 and spans(g):
        fmt = lambda t: [" ".join(s[a:b]), typ] for typ, a, b in spans(t)
        print("TWEET:", " ".join(s)); print(" gold:", fmt(g)); print(" pred:", fmt(p)); shown += 1
    if shown == 5: break
```

7. Save the model and predict on our own sentences

`save_pretrained` writes `config.json` (with our `id2label`), the weights and the tokenizer files. We then **reload from disk** to show that the saved artefact works on its own.

```

SAVE_DIR = "bert-ner-wnut16"
best_trainer.model.save_pretrained(SAVE_DIR); tokenizer.save_pretrained(SAVE_DIR)
print("saved:", sorted(os.listdir(SAVE_DIR)))
tokenizer = AutoTokenizer.from_pretrained(SAVE_DIR)
model_bert = AutoModelForTokenClassification.from_pretrained(SAVE_DIR).eval().to(DEVICE)
print("reloaded; labels:", list(model_bert.config.id2label.values())[5:], "...")

TWEET_TOKEN_RE = re.compile(r"https?:\/\/\S+|www\.\S+|[@#]?w+(?:[-'&]\w+)*|(^w\s)")
def tweet_tokenize(text): return TWEET_TOKEN_RE.findall(text)
def extract(words, tags): return [{" ".join(words[a:b]), typ} for typ, a, b in spans(tags)]

my_sentences = [
    "Just landed in New York for the Taylor Swift concert at Madison Square Garden tonight !",
    "@elonmusk says Tesla will open a new Gigafactory in Texas next year",
    "Watching Stranger Things on Netflix while the Lakers lose again lol",
    "Harry Potter was a student living in London",
    "Albus Dumbledore went to the Disney World",
    "My new iPhone 15 keeps crashing , thanks Apple 🙄 http://bit.ly/abc123",
    "Gig 'em ! Texas A&M beat Alabama at Kyle Field in College Station",
]
words = [tweet_tokenize(s) for s in my_sentences]
Xo, Co, _ = encode(words)
lstm_own, bert_own = predict_tags(best_lstm, Xo, Co, words), bert_predict_words(words)
for s, w, lt, bt in zip(my_sentences, words, lstm_own, bert_own):
    print(f"\n» {s}\n      BERT      : {extract(w, bt)}\n      BiLSTM-CRF: {extract(w, lt)}")

```

8. Answers to the remaining questions

Q4. Why do we need tokenization of the data in our case? Neural networks work on **integer ids and vectors, not strings**, so tokenisation turns text into the units the model is trained on.

1. *LSTM*: the Keras `Tokenizer` builds a vocabulary and maps each (normalised) word to an index into the word2vec-initialised embedding matrix. Unseen words map to `<OOV>`, and sequences are padded to a fixed length with a mask.
2. *BERT*: the model can only read ids from **its own pre-training WordPiece vocabulary**. Using any other tokeniser would feed it meaningless ids. WordPiece also solves the open-vocabulary problem of tweets: a rare or misspelt word such as `#snapchatbreach` or `tmrw` is split into known pieces instead of becoming `<OOV>`. The tokenizer also adds `[CLS]` `[SEP]`, attention masks and padding.
3. Because labels are per **word** but BERT's inputs are per **sub-word**, tokenisation forces the **label alignment** step (first sub-token labelled, the rest `-100`) and the matching **re-joining** step at prediction time.

Q5. What other models can be used for this task?

- *Classical*: a CRF on hand-crafted features (word shape, prefixes and suffixes, gazetteers, Brown clusters), HMMs, MEMMs, and structured perceptrons or SVMs. The earliest WNUT systems were feature-based CRFs.
- *Neural: BiLSTM-CNN-CRF* (Ma & Hovy, 2016) or a BiLSTM with **character-level** embeddings, which is very useful for OOV-heavy tweets. Also ID-CNNs, and contextual embeddings such as **ELMo** or **Flair** fed into a BiLSTM-CRF.
- *Transformers: BERTweet* or **Twitter-RoBERTa** (pre-trained on tweets, and the strongest choice for this data), RoBERTa, **DeBERTa-v3**, XLM-R for multilingual tweets, DistilBERT or ALBERT for speed, **LUKE** (entity-aware), a **BERT + CRF** head, and span-based models (SpanBERT, biaffine NER).
- *Generative or LLM-based*: seq2seq NER (T5 or BART generating the entities), instruction-tuned LLMs with few-shot prompting, and **GLiNER** or UniversalNER for zero-shot extraction of arbitrary entity types.
- *Off-the-shelf*: spaCy or Stanza NER pipelines (trained on news data, so a domain gap is expected on tweets).

Q7. How does BERT expect a pair of sentences to be processed? Both sentences are packed into **one** input: `[CLS]` tokens of A `[SEP]` tokens of B `[SEP]`. **Segment ids** (`token_type_ids`) are 0 for `[CLS]`, sentence A and the first `[SEP]`, and 1 for sentence B and the final `[SEP]`. Each input vector is the sum of the **token, segment and position embeddings**. The `attention_mask` marks real tokens versus padding, and if the pair is too long, (`truncation="longest_first"`) trims the longer sentence. In pre-training this format served **Next Sentence Prediction**, and it is used for sentence-pair tasks (NLI, QA, paraphrase), where the `[CLS]` vector is classified. With the Hugging Face tokenizer this is simply `tokenizer(sentence_a, sentence_b)`:

```

pair = AutoTokenizer.from_pretrained(MODEL_NAME)("Snapchat was breached.", "Which company was hacked?")
print(pd.DataFrame({"token": pair.tokens(), "token_type_id": pair["token_type_ids"], "attention": pair["attention_mask"]})).

```

Q8. Why choose attention-based models over recurrent ones?

- **Direct long-range context**: self-attention links any two tokens in one step (path length $O(1)$). An RNN must carry information through every intermediate step, where it fades (vanishing gradients), and even a BiLSTM combines two separate one-directional summaries. In BERT every layer is **deeply bidirectional**.
- **Parallelism**: a recurrent layer must process tokens one after another ($O(n)$ sequential steps), whereas attention processes all tokens at once. That makes it much faster on GPUs and TPUs, and practical to pre-train on billions of words.

- **Transfer learning:** that scalability made large-scale pre-training possible. BERT arrives already knowing that *Snapchat* is a company-like word, so a 2k-tweet dataset only has to teach it the label scheme. Our LSTM had to learn everything from scratch, which shows in the test results, especially on unseen entities.
- **Sub-word input with contextual embeddings:** the same word gets different vectors in different contexts (word2vec gives one fixed vector per word).
- **Trade-offs:** attention costs $O(n^2)$ in sequence length and memory, needs a GPU, and the models are about 100× larger (110M parameters vs roughly 1–2M here). For very long or streaming inputs, or on-device use, RNNs and state-space models can still be preferable.

Q9. BERT versus a simple (vanilla) Transformer

	Original Transformer (Vaswani et al., 2017)	BERT (De
Architecture	Encoder–decoder: 6+6 layers, with cross-attention from decoder to encoder	Encoder only: 12 layers (base) or 24 (large); no decoder
Attention direction	Encoder is bidirectional; decoder is causal (masked, left-to-right)	Fully bidirectional in every layer
Training objective	Supervised sequence-to-sequence task (machine translation), trained from scratch	Self-supervised pre-training (Masked Language Modelling + Next Se
Input representation	Token embeddings + sinusoidal positional encoding	WordPiece token + learned position + segment embeddings; special (
Output use	Generates the target sequence token by token	Contextual vectors for each token (tagging and QA) and [CLS] (class
Typical use	Translation and generation	Understanding tasks: NER, classification, QA, NLI

In short, BERT is the **encoder stack** of the Transformer, **pre-trained** with a masked-LM objective to give deep bidirectional representations that can be **fine-tuned** for tasks such as NER. A "simple transformer" is trained for one task from scratch and generates text with a decoder.

9. Conclusion

- Tweets are a difficult NER domain: noisy tokens, rare entities, about half the test entity tokens unseen in training, and a shift in entity-type mix between train and test.
- **BiLSTM-CRF:** casing features and word dropout, which both target the OOV problem, gave by far the largest gains. Bidirectionality and the CRF matter much less at this data size. Its ceiling is limited because word2vec vectors learned from 2k tweets carry little world knowledge.
- **BERT** substantially outperforms the LSTM on the held-out test set, thanks to pre-training and sub-word tokenisation. Early stopping picks the best epoch automatically and saves compute, and the optimiser and learning rate affect results less than having pre-trained weights at all.
- **Next steps:** a Twitter-specific encoder (BERTweet), a CRF head on BERT, character-level features for the LSTM, merging the validation data back into training, and ensembling.